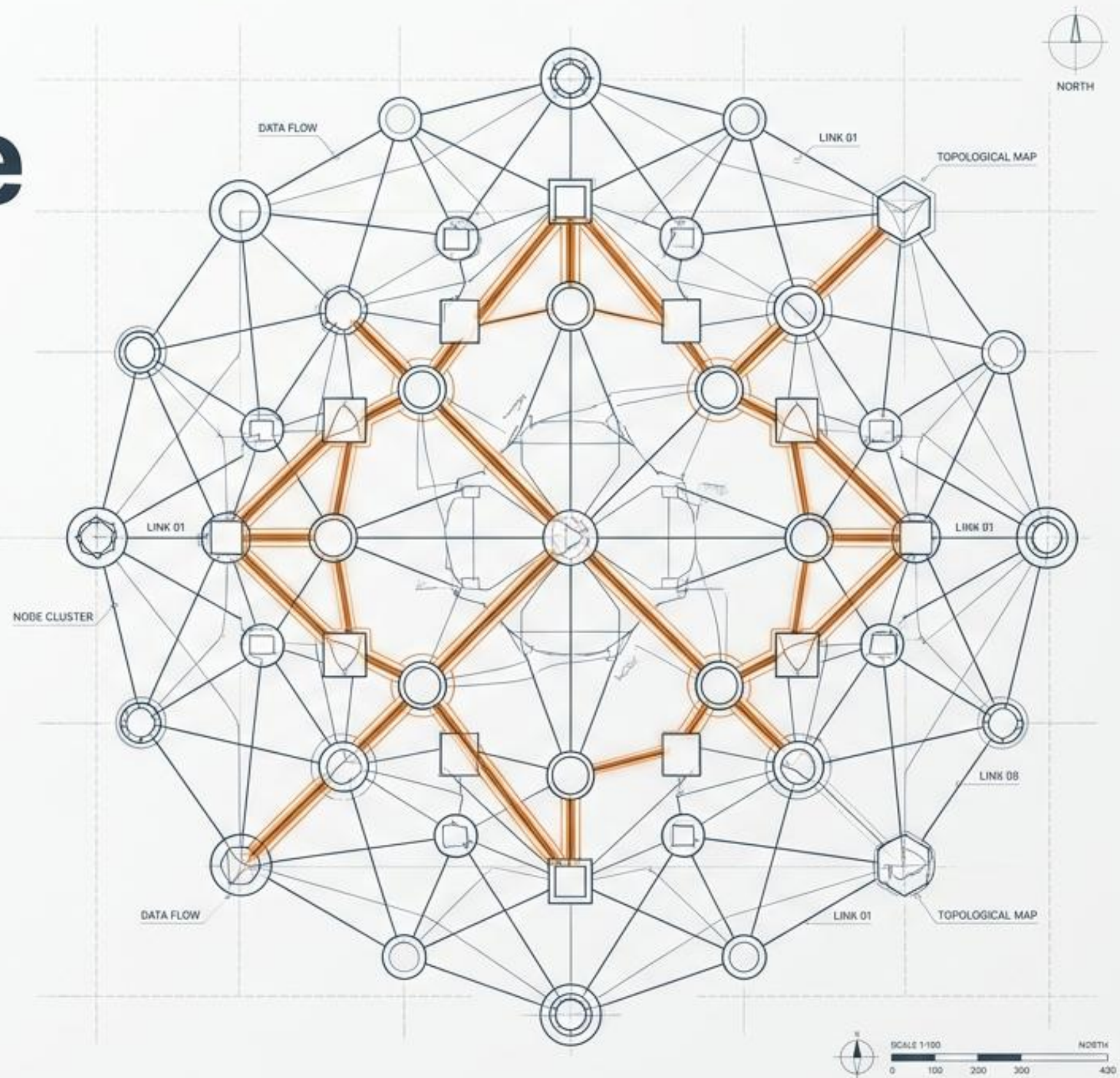


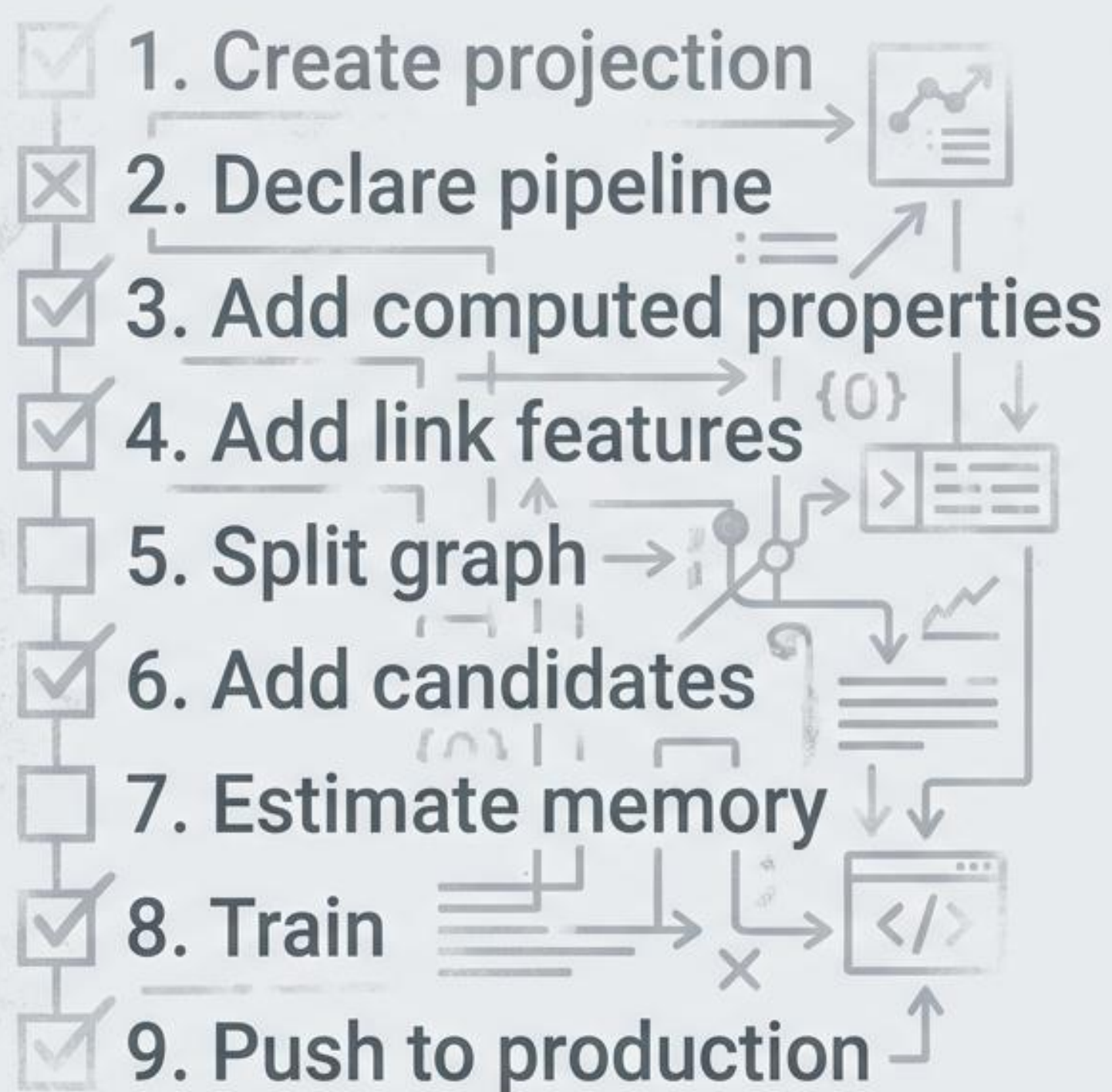
Graph Machine Learning

dr Zbigniew Galar

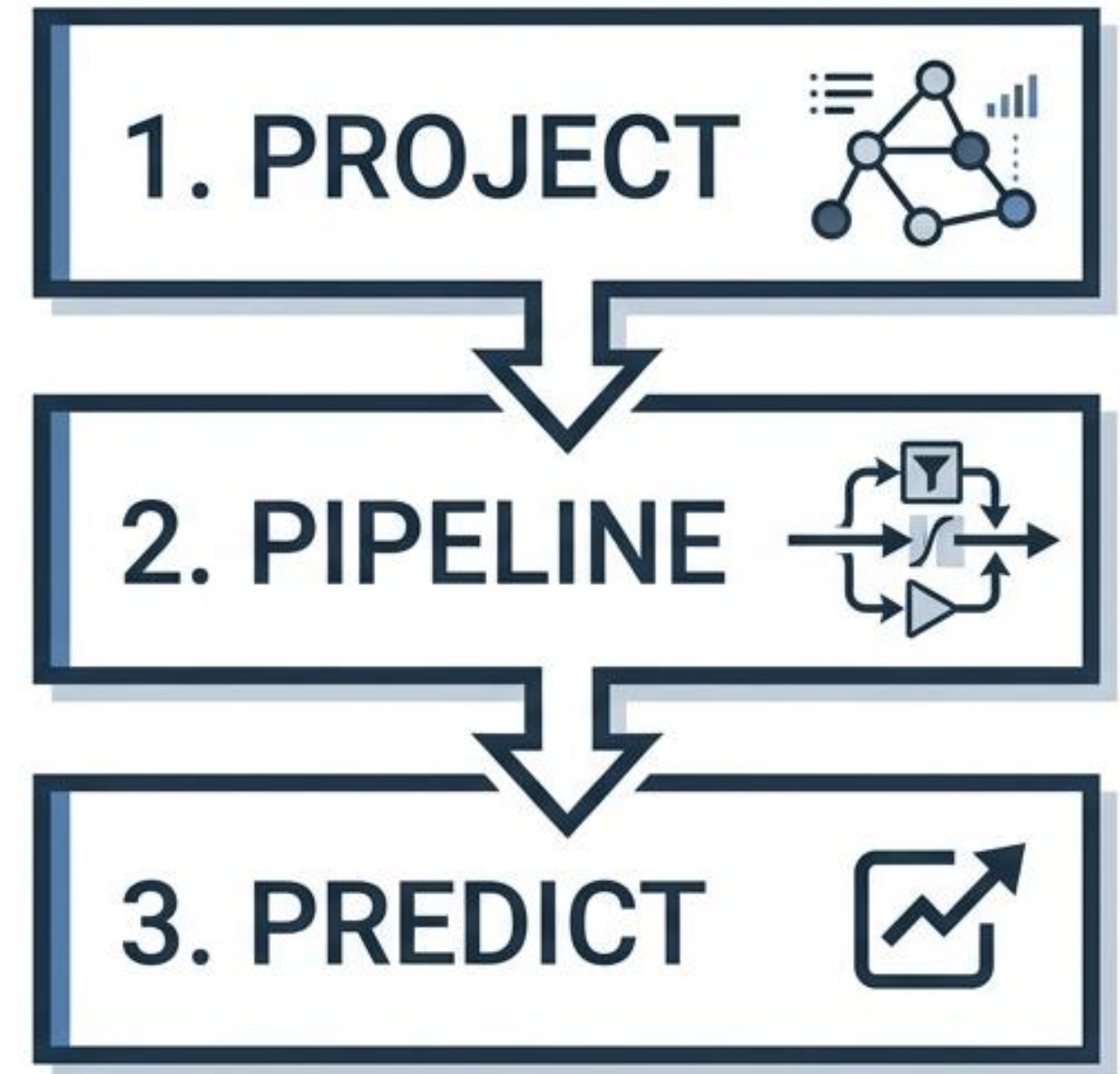
Declarative Pipelines &
Topological ML Blueprint



From manual extraction...

- ☒ 1. Create projection
 - ☒ 2. Declare pipeline
 - ☒ 3. Add computed properties
 - ☒ 4. Add link features
 - ☐ 5. Split graph
 - ☒ 6. Add candidates
 - ☐ 7. Estimate memory
 - ☒ 8. Train
 - ☒ 9. Push to production
- 

...to declarative configuration



Topological machine learning instinctively feels like massive effort. But within a graph-native environment, functionality is built-in. You do not implement the math; you simply declare the pipeline's configuration.

Defining the target topology

Before modeling, we execute a single pretraining step: collapsing indirect cinematic relationships into direct topological connections.

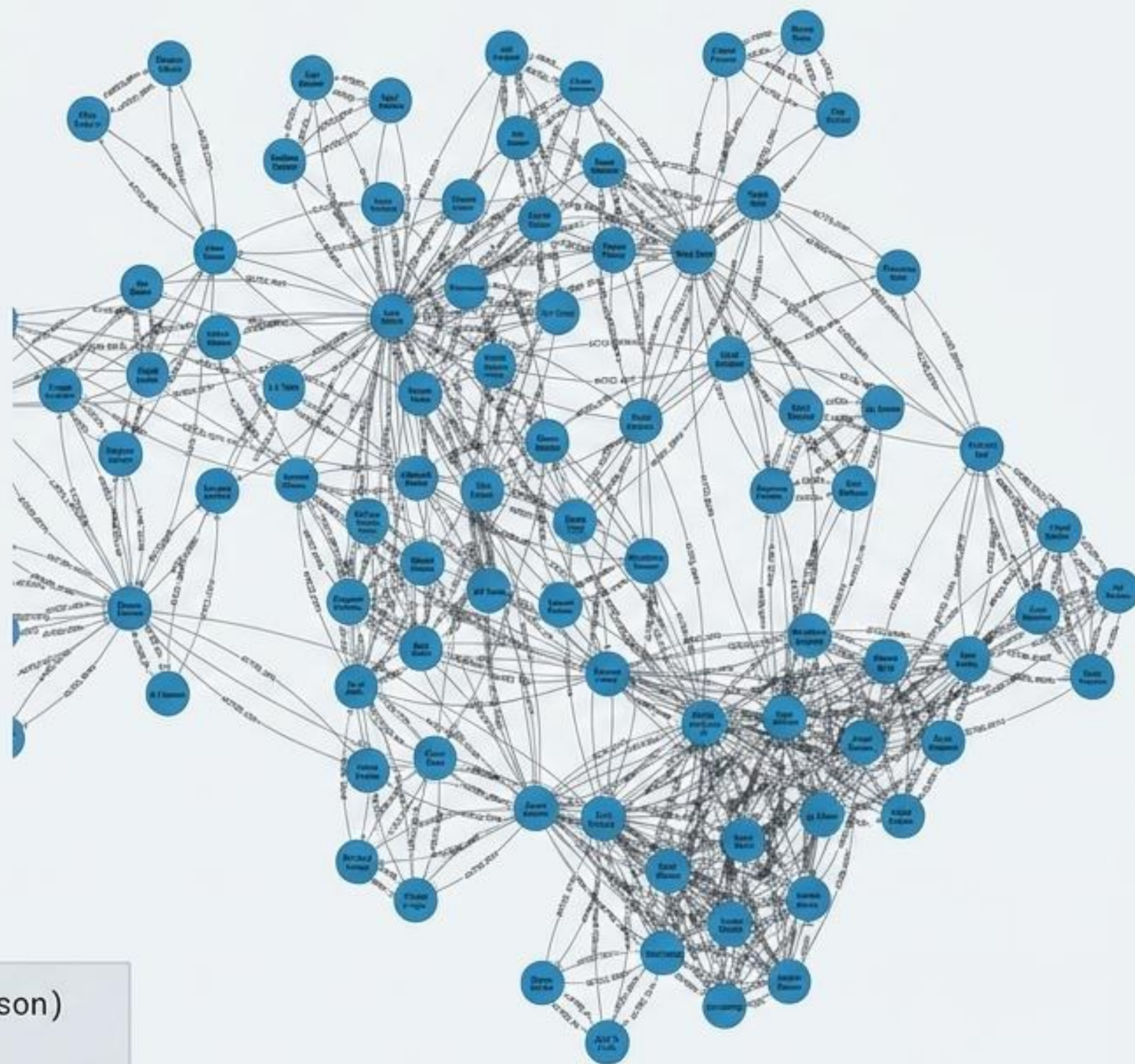
Before & After



After

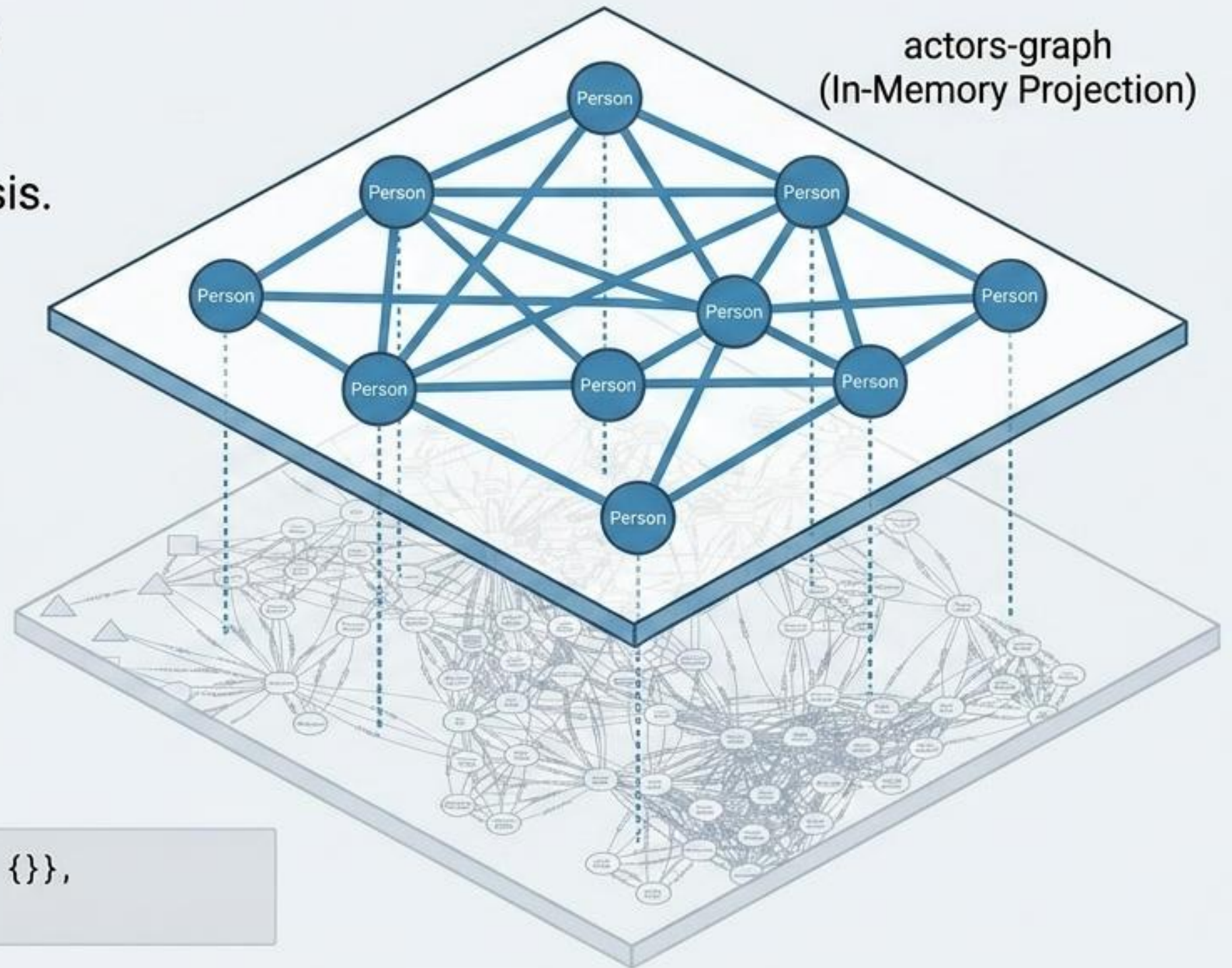


```
MATCH (a:Person)-[:ACTED_IN]->(:Movie)<-[:ACTED_IN]-(b:Person)
MERGE (a)-[:ACTED_WITH]->(b)
```



Isolating the topological signal via graph projection

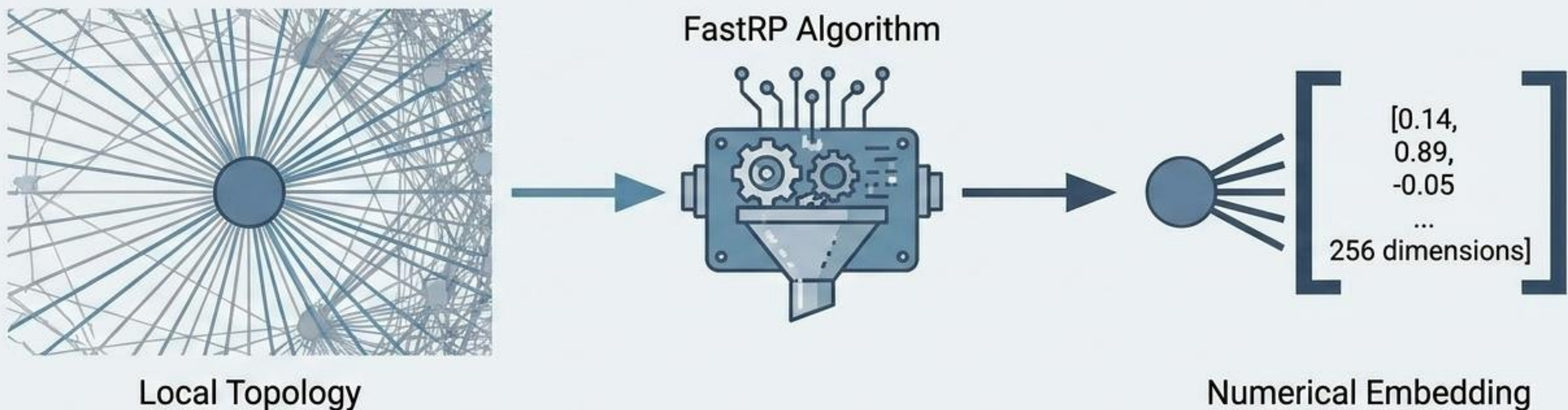
Machine learning models require focus. We create an in-memory projection containing only the specific nodes and undirected relationships required for topological analysis.



```
CALL gds.graph.project('actors-graph', {Person: {}},  
  {ACTED_WITH: {orientation: 'UNDIRECTED'}})
```

Encoding local topology as a numerical embedding

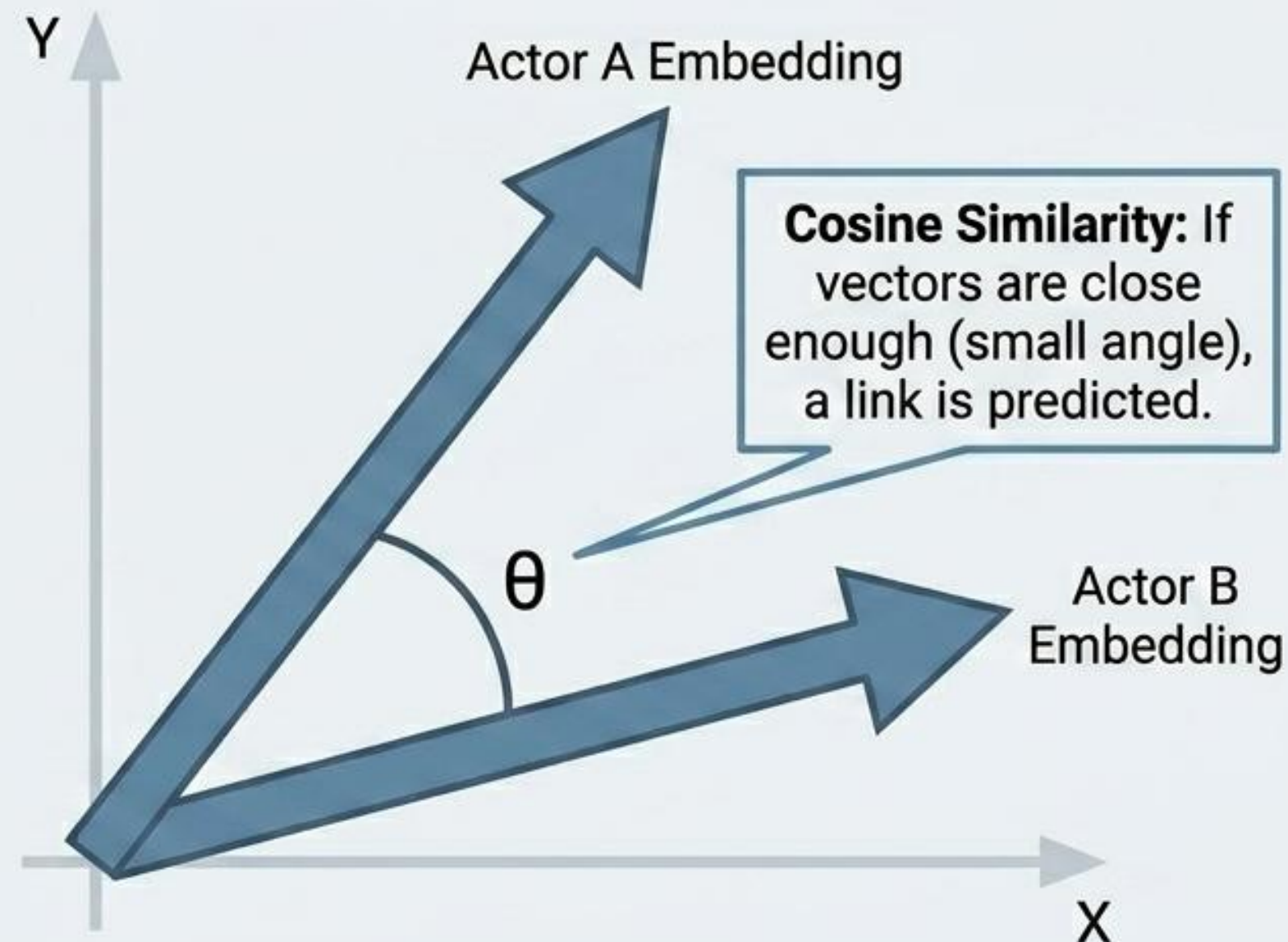
A node's position in the network contains predictive power. We configure the pipeline to inject the FastRP algorithm, generating a 256-dimension numerical encoding representing each actor's unique place in the topology.



```
CALL gds.beta.pipeline.linkPrediction.addNodeProperty('actors-pipeline', 'fastRP', {  
  mutateProperty: 'embedding',  
  embeddingDimension: 256,  
  randomSeed: 42})
```

Calculating closeness with link features

Link features join a node pair together for topological ML. By adding a cosine feature, the pipeline compares the mathematical angles of two embeddings to determine structural similarity.



```
CALL gds.beta.pipeline.linkPrediction.addFeature('actors-pipeline', 'cosine', {  
  nodeProperties: ['embedding']  
})
```

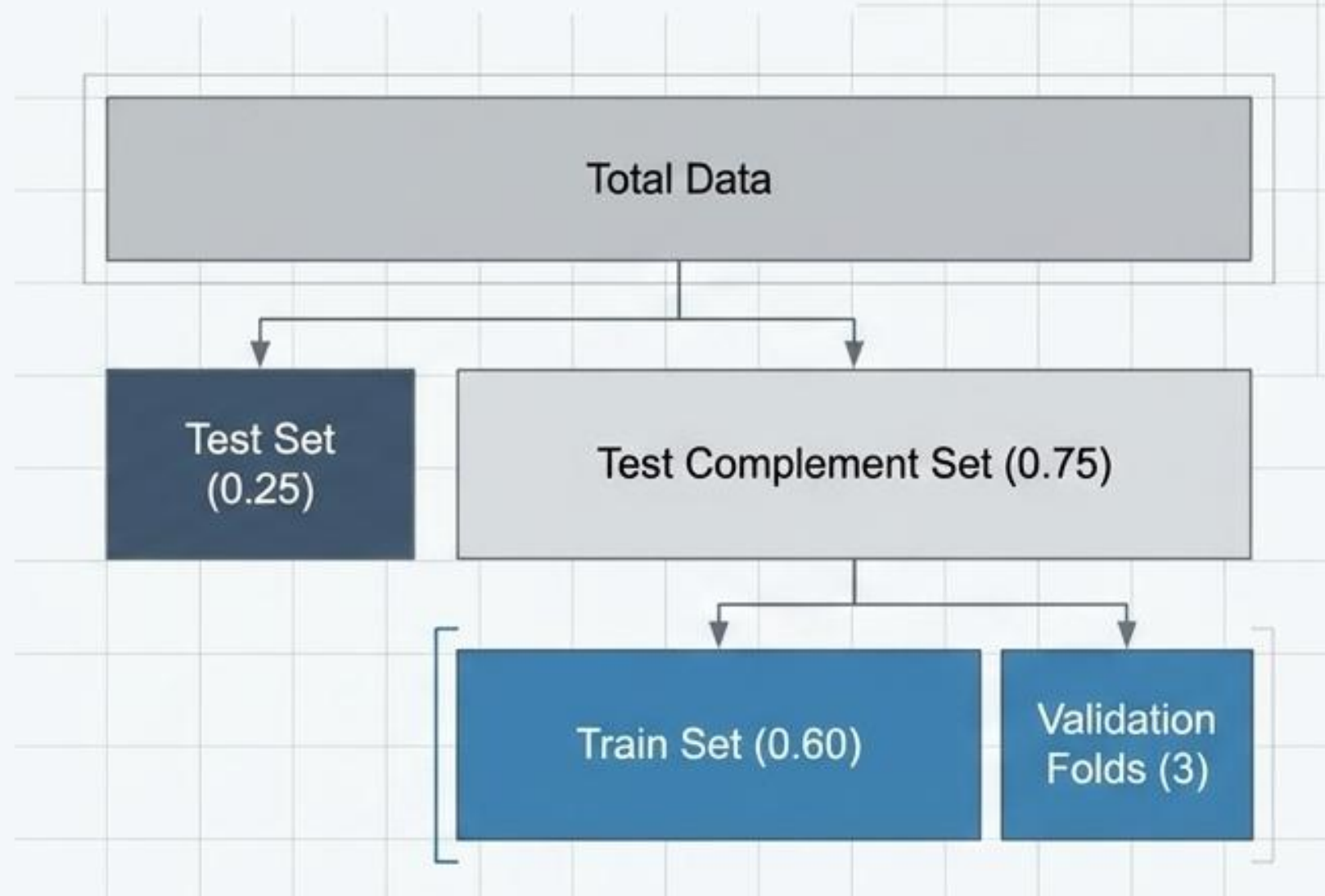
The declarative configuration matrix

The pipeline is agnostic to the underlying math. You simply register the desired mathematical feature extractors and model candidates, and the engine handles execution.

		Link Feature Algorithms		
		Cosine Similarity	Hadamard Product	L2 Euclidean
Model Candidates	Logistic Regression	Active Configuration		
	Random Forest			
	Multilayer Perceptron			

Partitioning the topology for validation

Feeding examples into the pipeline is automated. We declare the exact fraction of the graph reserved for testing, and the proportion of the remaining data used to train.



```
CALL gds.beta.pipeline.linkPrediction.configureSplit('actors-pipeline', {
  testFraction: 0.25,
  trainFraction: 0.6,
  validationFolds: 3
})
```

Auto-tuning and pre-flight resource checks

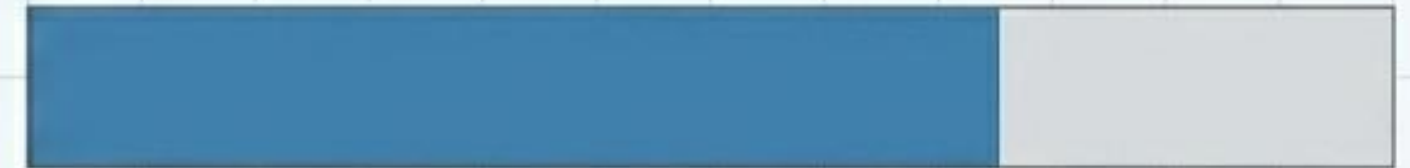
Before committing computational cycles, the pipeline readies models for automated hyperparameter tuning and safely verifies that the underlying hardware contains sufficient memory.

Auto-Tuning Engine



Resource Estimator

YIELD requiredMemory



Simulated RAM

```
CALL gds.beta.pipeline.linkPrediction.train.estimate('actors-graph', {  
  pipeline: 'actors-pipeline',  
  modelName: 'actors-model',  
  targetRelationshipType: 'ACTED_WITH'  
}) YIELD requiredMemory
```

Executing the training phase

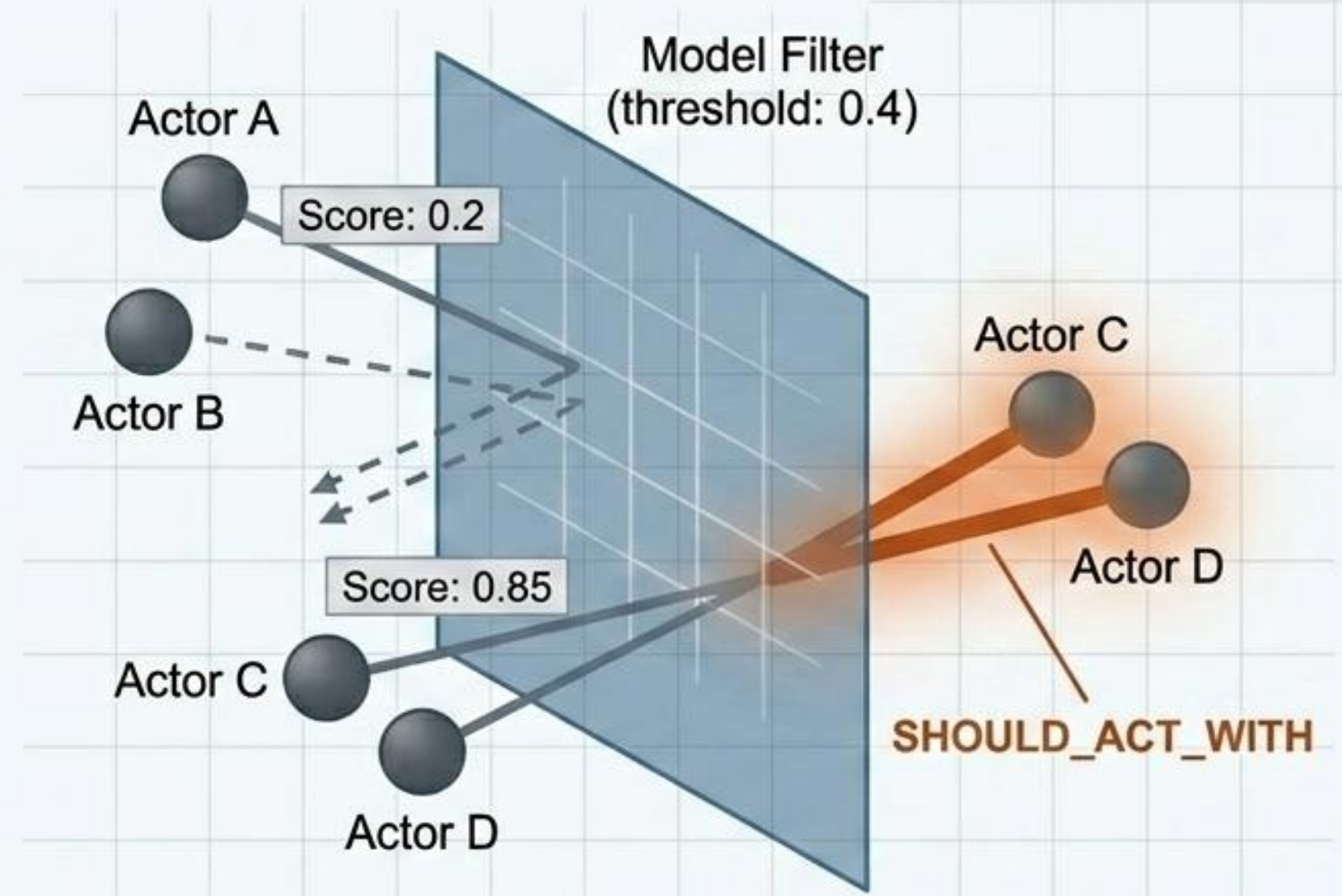
Instantiating the configuration executes the training phase. The output is a registered model artifact alongside comprehensive AUCPR metrics allowing data scientists to reason about prediction quality.



```
CALL gds.beta.pipeline.linkPrediction.train(...) {  
  pipeline: 'actors-pipeline',  
  modelName: 'actors-model',  
  YIELD modelInfo, modelSelectionStats  
}
```

Applying predictive intelligence via mutation

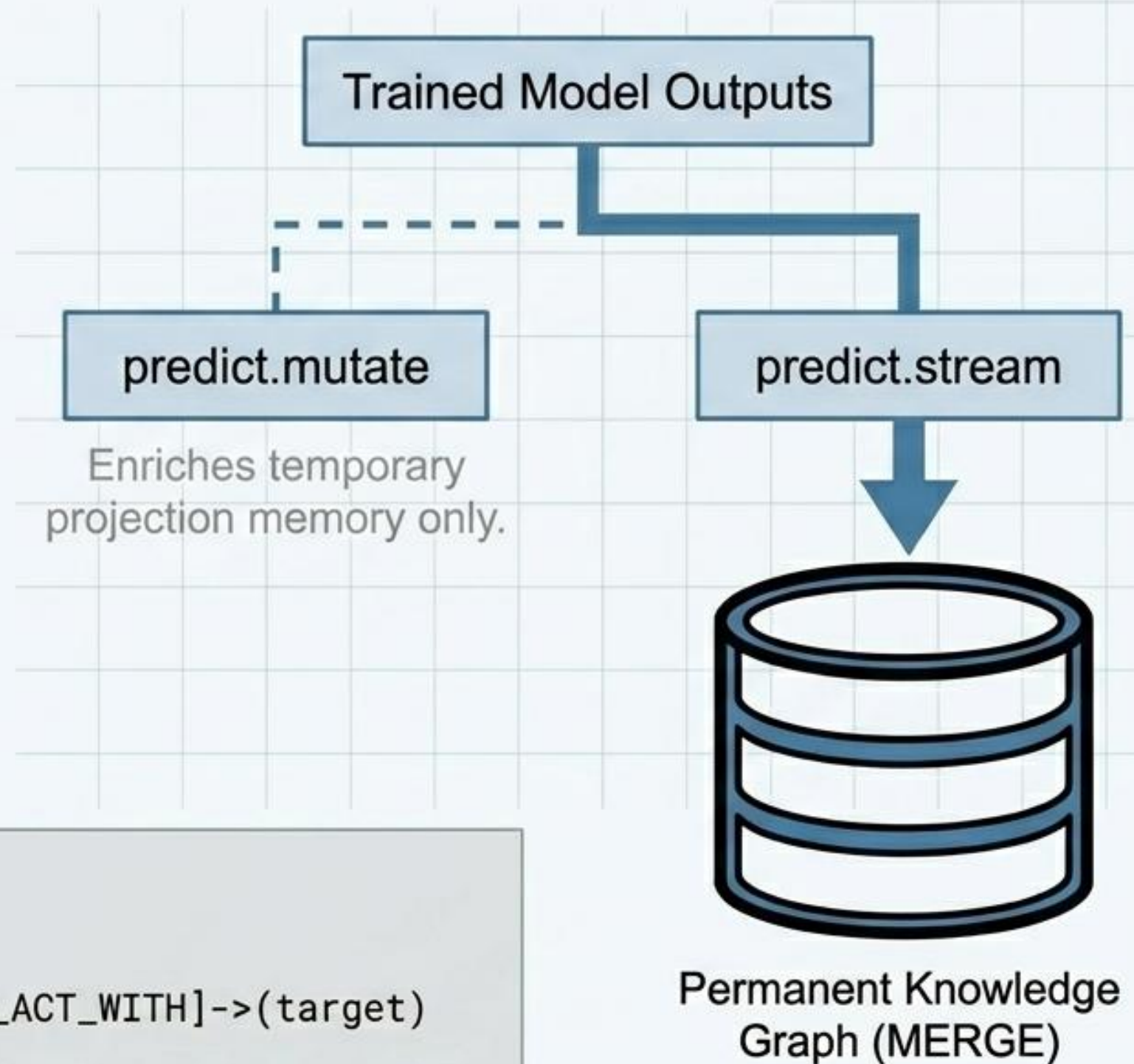
The trained model acts as a topological filter. Actor pairs are evaluated; if their structural confidence score clears our defined threshold, the algorithm draws an explicit, predictive relationship directly into the projection.



```
CALL gds.beta.pipeline.linkPrediction.predict.mutate('actors-input', {  
  modelName: 'actors-model',  
  mutateRelationshipType: 'SHOULD_ACT_WITH',  
  topN: 20,  
}) threshold: 0.4
```

Persisting predictions back to the foundation

Projections are ephemeral. To permanently enrich the source knowledge graph, we stream predicted relationships out of the model and seamlessly MERGE them back into the base topology.

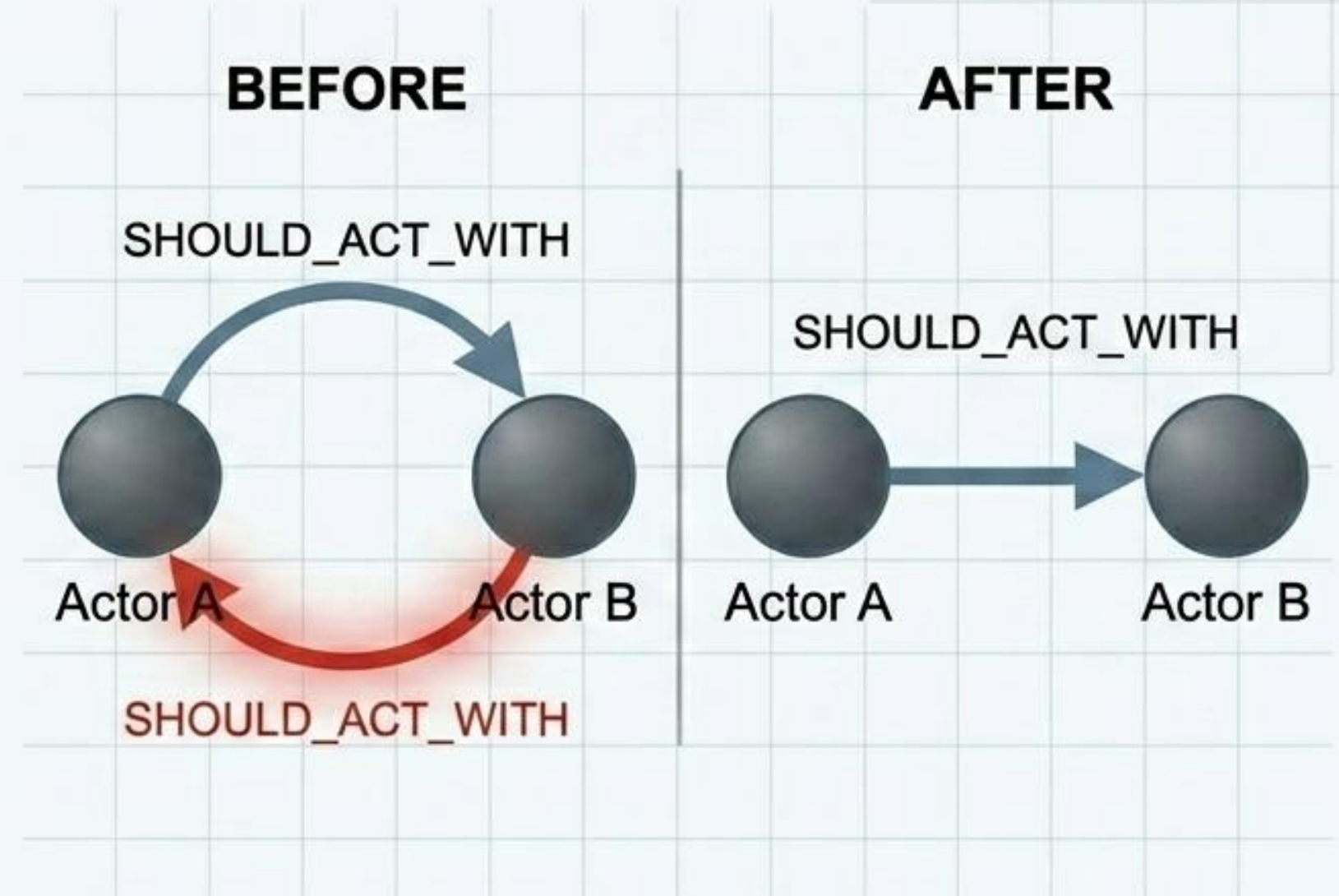


```
CALL gds.beta.graph.relationships.stream(...)
YIELD sourceNodeId, targetNodeId

WITH gds.util.asNode(...) MERGE(source)-[:SHOULD_ACT_WITH]->(target)
```

Normalizing the enriched knowledge graph

Because complementary acting recommendations are inherently symmetric, storing reciprocal edges is redundant. A simple Cypher pass deletes inverse duplicates, ensuring structural hygiene.



```
MATCH (a:Person)-[:SHOULD_ACT_WITH]->(b:Person)-[d:SHOULD_ACT_WITH]->(a)
WHERE id(a) > id(b)
DELETE d
```

Extracting intelligence with standard queries

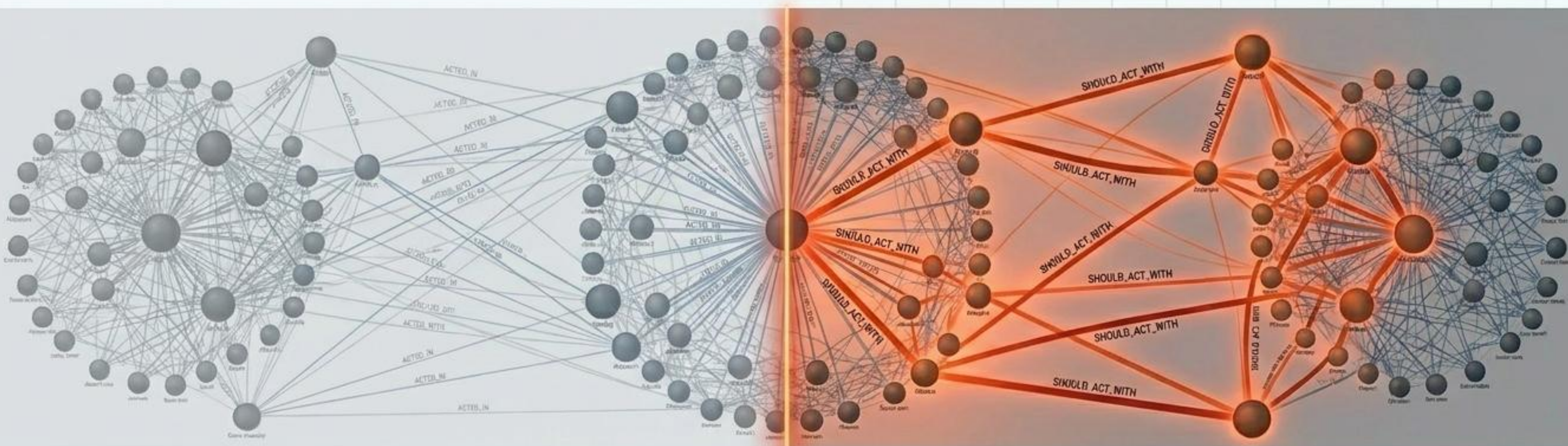


Once wrangling is complete, the knowledge graph is forever enriched. Extracting a highly complex machine learning recommendation is now reduced to the simplest possible pattern-matching query.

```
MATCH (a:Person)-[:SHOULD_ACT_WITH]->(b:Person)
RETURN a.name, b.name
RETURN a.name, b.name

MATCH (a:Person)-[:SHOULD_ACT_WITH]->(b:Person)
RETURN a.name, b.name
```

Actor Recommendations	
Tom Hanks	Meg Ryan
Keanu Reeves	Carrie-Anne Moss
Harrison Ford	Sean Connery



The Past: Implicit Historical Facts (ACTED_IN, ACTED_WITH)

The Future: Explicit Predictive Intelligence (SHOULD_ACT_WITH)